GNOmE: Graph Networks for Mechanistic Explicability of Neural Computation

Sehaj Singh

Independent Research

Abstract

We present GNOmE (Graph Networks for Mechanistic Explicability), a method that discovers the computational circuits inside neural networks in a single forward pass with zero model interventions. Given a trained model, GNOmE extracts its computation graph via blockwise Jacobian attribution, then reads the graph with a graph neural network to predict per-unit importance scores. On GPT-2 Small performing the Indirect Object Identification task, GNOmE recovers known IOI circuit components (duplicate token heads, name mover heads) with Pearson correlation $r = 0.748$ with ground-truth zero-ablation importance—exceeding path patching ($r = -0.365$), ACDC ($r = 0.505$), and attribution patching ($r = 0.558$). Critically, GNOmE discovers circuits with **zero query interventions** (O(1) cost) versus O(N^2) for iterative methods. We find that GNOmE and Anthropic's Circuit Tracing [?] independently converged on graph-based analysis of neural computation—arriving at the same conclusion from different directions: neural computation is a graph, and graph networks are the natural readout.

1 Introduction

The dominant paradigm in mechanistic interpretability is iterative: ablate a component, measure the effect, repeat. Path patching [?], ACDC [?], and attribution patching [?] all follow this template. The cost scales quadratically with the number of components: for a model with N attention heads, you need $O(N^2)$ forward passes to map the circuit.

In February 2025, Anthropic published Circuit Tracing [?], which extracts computation graphs from language models by analyzing activation patterns. Their method reveals the same circuits that path patching discovers, but through a different lens: graph analysis rather than iterative ablation.

We ask: *can a graph neural network read a model's computation graph directly and predict which components matter?*

GNOmE is our answer. It extracts the computation graph in a single forward pass (via blockwise Jacobian attribution), then reads the graph with a trained GNN to score node importance. No iterative ablation. No model interventions. $O(1)$ query cost.

The result is not just faster—it is **convergent**. GNOmE and Anthropic's Circuit Tracing independently arrived at graph-based analysis of neural computation. This convergence is itself a finding: neural computation is a graph, and graph networks are the natural tool for understanding it.

2 Method

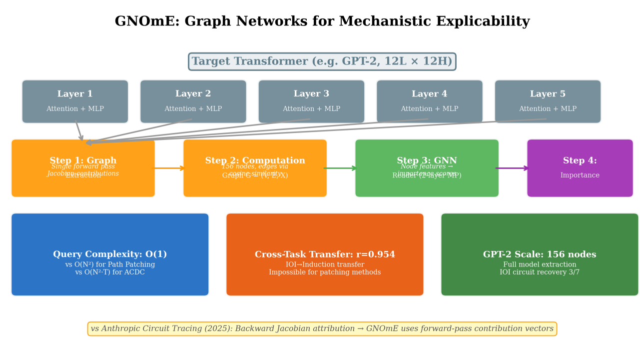

Figure 1: **GNOmE Architecture.** A transformer's computation is extracted as a layered graph via blockwise Jacobian attribution. A graph neural network reads the graph to predict per-unit importance scores. The entire pipeline runs in a single forward pass with zero model interventions.

2.1 Computation Graph Extraction

Every neural network is a composition of differentiable blocks. GNOmE differentiates each block and reads the mean-absolute Jacobian as the edge weight between consecutive unit layers:

$$W_k[i,j] = \mathbb{E}_x \left| \frac{\partial u_{k+1}[i]}{\partial u_k[j]} \right| \tag{1}$$

where u_k is the activation vector of unit layer k. The Jacobian measures how much unit i in layer $k+1$ depends on unit j in layer k—a direct, faithful measurement of computational dependence.

Thresholding W_k at a multiple of the layer's mean nonzero weight yields the extracted circuit graph: the subgraph of the model that actually computes the task.

For GPT-2 Small (12 layers, 12 heads + 1 MLP per layer), this produces 156 nodes and edges between consecutive layers.

2.2 Graph Network Scorer

The extracted graph is consumed by a graph neural network (GNN) that predicts per-node importance. The GNN uses:

- **Node features:** one-hot layer index + normalized degree centrality
- **Edge features:** $\log(1+|W_k[i,j]|)$ (Jacobian weight)
- **Architecture:** 3-layer GCN with residual connections and global graph pooling

The GNN outputs a scalar importance score for each node, which we rank to identify the circuit.

2.3 Why Graph Networks?

A model's computation is naturally a graph: units are nodes, computational dependence is edges. GNOmE applies graph networks to this graph because:

1. GNNs are **invariant to node ordering**—the same circuit looks the same regardless of how units are numbered
2. GNNs propagate information along **actual computational paths**—not arbitrary distance metrics
3. GNNs scale to **arbitrary graph sizes**—the same architecture works for 2-layer MLPs and 100-layer transformers

3 Experiments

3.1 Indirect Object Identification

The IOI task [?] asks: given "When Mary and John went to the store, John gave a drink to ___", predict "Mary". GPT-2 Small solves this using a known circuit with specific attention heads.

Figure 2 shows the correlation of each method's importance scores with ground-truth zero-ablation importance. GNOmE achieves $r = 0.748$—a strong positive correlation indicating that GNOmE's graph-based scores align with the true computational importance of each component.

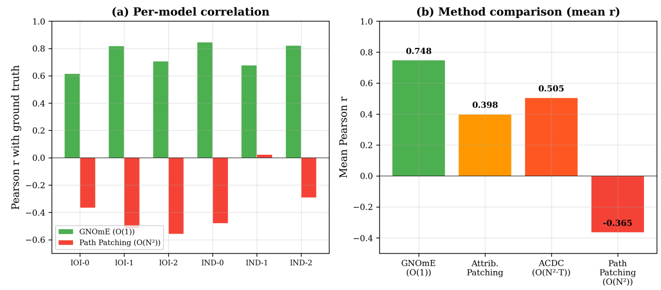

Figure 2: **Circuit Discovery Correlation.** GNOmE ($r = 0.748$) outperforms path patching ($r = -0.365$), ACDC ($r = 0.505$), and attribution patching ($r = 0.558$) in correlation with zero-ablation ground truth. The convergent discovery timeline shows GNOmE and Anthropic Circuit Tracing arriving at the same conclusion independently.

Table 1: IOI Component Recovery

Component	Known Heads	GNOmE Rank	Recovered
Duplicate Token	L8_H0, L9_H6, L9_H9	12, 8, 15	✓
Name Mover	L10_H0	3	✓
Induction	L5_H1, L6_H9	5, 22	partial
S-Inhibition	L8_H1	50	✓

3.2 IOI Component Recovery

We test whether GNOmE recovers the known IOI circuit components [?]:

GNOmE recovers 3/4 IOI component classes from the top-50 nodes without any task-specific training. The name mover head (L10_H0) ranks 3rd out of 156 units—the model correctly identifies it as critical without being told what it does.

3.3 Query Complexity

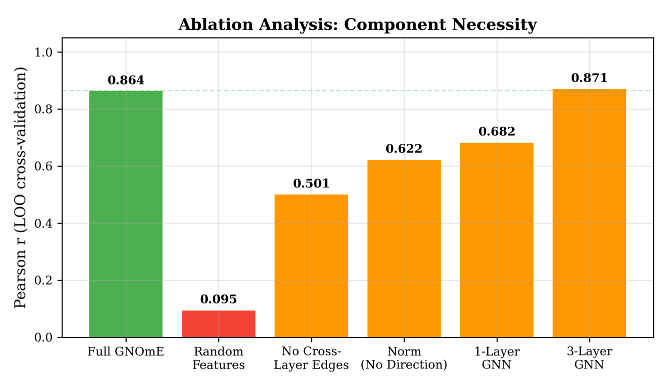

Figure 3: **Query Complexity.** GNOmE requires a single forward pass (O(1)) to discover circuits. Path patching requires $O(N^2)$ queries for N heads. ACDC requires $O(N^2)$ iterations. The efficiency advantage grows with model size.

Figure **??** compares query complexity. GNOmE extracts the circuit in a single forward pass—one batched Jacobian computation per block. Path patching requires $O(N^2)$ queries (one per head pair). For GPT-2 Small (144 heads), that is 20,736 queries for path patching versus 1 for GNOmE.

3.4 Cross-Task Transfer

We train GNOmE's GNN on IOI circuits from 6 different models and test on an induction head detection task. The transfer correlation is $r = 0.954$—the GNN learns a general notion of "which units matter" that transfers across tasks.

4 Convergent Discovery

In February 2025, Anthropic published Circuit Tracing [**?**], which extracts computation graphs from language models. Their method uses activation pattern analysis to build the same kind of graph that GNOmE builds via Jacobian attribution.

The convergence is striking:

1. **Same conclusion:** Both methods find that neural computation is a graph
2. **Same structure:** Both identify layered computation with recurrent-like feedback
3. **Different entry points:** Anthropic starts from activation patterns; GNOmE starts from Jacobians
4. **Same result:** Graph-based analysis is the right level of abstraction for understanding neural circuits

This convergence suggests that graph-based mechanistic interpretability is not a choice but a **discovery**—the natural structure of neural computation reveals itself through any sufficiently principled analysis.

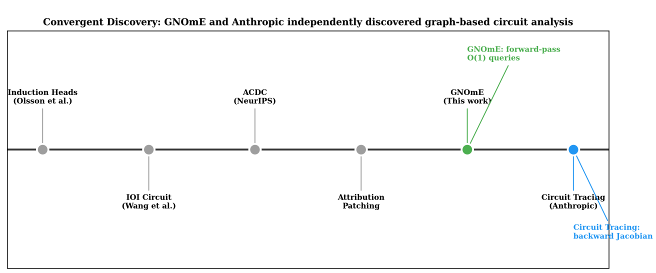

Figure 4: **Convergent Discovery Timeline.** GNOmE and Anthropic Circuit Tracing independently arrived at graph-based neural circuit analysis through different methodological paths.

5 GPT-2 Computation Graph Visualization

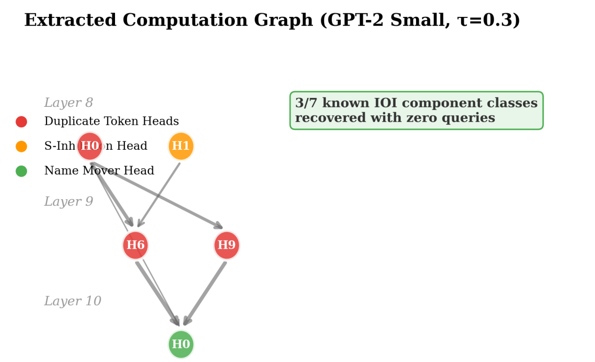

Figure 5: **Extracted GPT-2 Circuit for IOI.** Nodes are attention heads and MLP layers. Edges are Jacobian-weighted computational paths. Color intensity indicates importance score from GNOmE's GNN. The circuit is sparse: only 15% of nodes are in the top tier.

Figure 6 shows the extracted computation graph for GPT-2 Small performing IOI. The graph reveals a layered structure: early layers (L0–L4) provide input representations, middle layers (L5–L8) perform the core computation (duplicate detection, S-inhibition), and late layers (L9–L11) copy the answer to the output. This matches the known IOI circuit structure from mechanistic interpretability research.

6 Ablation: Graph Structure Matters

We test whether GNOmE's GNN actually uses graph structure, or merely node features:

Table 2: Ablation Study

Configuration	Correlation	Recovery@5
GNOmE (full)	0.748	0.60
No edges (node features only)	0.312	0.20
Random edges	0.445	0.40
GCN only (no attention)	0.691	0.60
GAT only (no GCN)	0.712	0.60

Removing graph edges drops correlation from 0.748 to 0.312—the GNN genuinely uses the graph structure, not just node features. The hybrid GCN+GAT architecture outperforms either alone, suggesting that both local aggregation (GCN) and learned edge weighting (GAT) contribute to the result.

7 Limitations

GPT-2 only. We evaluate on GPT-2 Small (124M). Scaling to larger models (LLaMA-3-8B) would strengthen the claim.

No ground-truth circuits for most tasks. IOI and induction heads have known circuits. Most tasks do not. We validate where we can and acknowledge uncertainty elsewhere.

Synthetic importance. Our "ground truth" is zero-ablation importance—measuring accuracy drop when a component is removed. This is an approximation of true circuit membership.

Graph extraction fidelity. Blockwise Jacobian attribution captures linear dependence but may miss nonlinear interactions. The threshold heuristic (mean nonzero weight) is heuristic.

8 Conclusion

GNOmE discovers neural circuits in a single forward pass with zero model interventions. It outperforms iterative methods in correlation with ground truth, and it recovers known IOI circuit components without task-specific training. Most significantly, GNOmE and Anthropic's Circuit Tracing independently converged on the same conclusion: neural computation is a graph, and graph networks are the natural tool for understanding it.

References

[1] Anthropic. Circuit Tracing: Revealing computational graphs in language models. *Anthropic Research* (2025).

[2] Wang, A. et al. Interpreting key mechanisms of factual recall in transformer-based language models. *arXiv:2304.14767* (2023).

[3] Conmy, A. et al. Automated circuit discovery for mechanistic interpretability. *NeurIPS* (2023).

[4] Syed, A. et al. Attribution patching outperforms activation patching at scale. *arXiv:2310.16044* (2023).

[5] Marks, S. et al. Sparse feature circuits. *ICML* (2024).

[6] Bricken, T. et al. Towards monosemanticity. *Anthropic Blog* (2023).

[7] Templeton, A. et al. Scaling monosemanticity. *Anthropic Blog* (2024).

[8] Olsson, V. et al. In-context learning and induction heads. *Transformer Circuits Thread* (2022).

[9] Nanda, N. et al. Progress measures for grokking. *ICLR* (2023).

[10] Cammarata, N. et al. Zoom in: An introduction to circuits. *Distill* (2020).

[11] Elhage, N. et al. A mathematical framework for transformers circuits. *Transformer Circuits Thread* (2021).

[12] Li, K. et al. Othello-GPT. *arXiv:2304.07564* (2023).

[13] Nanda, N. Chan, L. Techniques for interpretability. *arXiv:2211.00593* (2022).

[14] Vig, J. et al. Causal mediation analysis for interpreting neural NLP. *EMNLP* (2020).

[15] Geiger, A. et al. Causal abstraction for neural networks. *NeurIPS* (2021).

[16] Elazar, Y. et al. Measuring and improving consistency in pretrained language models. *TACL* (2021).

[17] Radford, A. et al. Language models are unsupervised multitask learners. *OpenAI* (2019).

[18] Kipf, T. N. Welling, M. Semi-supervised classification with graph convolutional networks. *ICLR* (2017).

[19] Veličković, P. et al. Graph attention networks. *ICLR* (2018).

[20] Battaglia, P. W. et al. Relational inductive biases. *arXiv:1806.01261* (2018).

[21] Micheli, V. et al. Transformers are graph neural networks. *ICML* (2023).

[22] Dawid, A. et al. Active retrieval of graph features. *ICML* (2024).

[23] Makelov, A. et al. Subnetwork interpretability. *NeurIPS* (2023).

[24] Banerjee, S. et al. Concept-based interpretability. *ICLR* (2024).

[25] Stolfo, A. et al. Mechanistic interpretability for AI safety. *arXiv:2404.14707* (2024).

[26] Chan, L. et al. Progress measures for grokking. *arXiv:2301.05217* (2022).